PLATFORM BUILT IN

lifepal

PRESENTED TO

TRIPADVISOR

DATA PLATFORM · SENIOR DATA ENGINEER

DATA ENGINEERING · FIRESIDE CHAT · NEXUS BEST PRACTICES FROM LIFEPAL

Nexus

Every event counted once, on time, and checked

Four years leading the data platform at Lifepal, a high-traffic consumer marketplace: Kafka and Debezium streams next to batch sources, a config-driven framework other engineers built on, SLAs, 500+ quality checks and a root-cause review after every incident. This pack applies those practices to a travel site's data on Tripadvisor's listed stack, Kafka → stream processing → Spark → Snowflake and BigQuery, in a runnable repository with synthetic data. Every run figure comes from that run; every Lifepal figure is from my CV.

PRESENTED BY
R. Firman Insan Muhammad (Vim)
Senior Data Engineer · London

TO
Data Platform
Tripadvisor

ROLE
Senior Data Engineer
London, remote / hybrid

STACK
Kafka · Flink · Spark
Snowflake · BigQuery · Java · Python

CONTENTS

What I'd like to cover

Ten minutes on what I've built, then the parts of the role I'd own first. Every page points to working code.

01	**Measured outcomes**	What changed at Lifepal, in numbers
02	**The role, line by line**	Where I've done it before, and where I haven't
03	**Six streaming problems**	Planted in the topics, caught in the run
04	**The stream job**	Event time, watermarks, lateness, dead letters
05	**Stream first, batch checks**	Reconciliation, record for record; Spark agrees
06	**Snowflake and BigQuery from one contract**	Governance as generated code
07	**SLAs and an incident**	Detection, root cause, postmortem
08	**Java**	A twin of the job, tested against the reference
09	**My first 90 days**	How I'd start in Data Platform

What changed at Lifepal

Figures from my CV and production work, described at the level I can share publicly.

99.7%
reporting latency cut: 24 h → under 5 min

75%
nightly batch cut: 6 h → 90 min by tuning Spark and SQL

$11k
monthly platform cost against a $19k forecast

−60%
data-trust escalations: SLAs, 500+ checks, anomaly holds

−40%
duplicate records

How: one governed lakehouse out of fragmented batch jobs and legacy ingestion; Kafka + Debezium CDC next to batch from APIs and partner files; a config-driven Python framework; Kimball models and a dbt semantic layer for five teams.

Team: led 3–5 engineers; two promoted to Senior; onboarding halved from 4 weeks to 2. An MCP server for governed agent access and a RAG copilot with 1,000+ outcomes a day.

How my experience maps to the role

The job description's asks, what I've done, and where it shows in this repository.

THE ROLE ASKS FOR	WHAT I'VE DONE	HERE
ETL/ELT across Snowflake and BigQuery	Snowflake + dbt at Deloitte; Databricks + dbt at Lifepal	warehouse/ · warehouse.py
Own pipelines: SLAs, monitoring, root cause	SLAs, freshness and volume alerts, runbooks, RCA after every incident	sla.py · oncall.html
Streaming and batch: Flink, Spark, Kafka	Kafka + Debezium with Spark; 6 h → 90 min batch	stream.py · spark_batch.py
Production Java and Python	Python for ten years; Java is my ramp	java/WindowJob.java
Shared frameworks, governance tooling	Config-driven ingestion; PII masking, RBAC, lineage	contracts/ · governance.sql
GenAI, analytics, ML	RAG copilot; MCP server; feature pipelines	Fireside story
Flink, specifically	Not in production. Semantics here in Python and Java; Flink SQL in flink/	flink/*.sql

What Kafka really delivers

A week of a travel site: 26,426 records on 13 partitions. Each problem is planted, handled and tested.

PROBLEM	IN THE RUN	WHAT HANDLES IT
SDK retries	112 events re-sent, same event_id, new offset	Dedupe on id, 24 h checkpointed state: 112 dropped
An offline upload	192 iOS events, 2–8 hours late	Side output past an hour's lateness; the batch explains 159 windows
A schema change	app 7.4: price {amount, currency}, 302 records	One reader upcasts v1 and v2; unknown currency is a dead letter
A poison pill	partition 3, offset 171: 57 bytes of truncated JSON	74 restarts, paged at 14:35; fixed with the DLQ
A scraper	1,515 views in an hour that has 29.0	Hour held (2σ and ×2); 3 users removed; released
Erasure, then a stale version	an outbox replay re-sends version 3 after the tombstone	Version rule: 1 rejected; deleted in both warehouses

Event time, decided by the watermark

The watermark is the minimum over active partitions, less the out-of-orderness. A quiet partition stops holding it back; when every partition is quiet, processing time carries it.

$$\text{wm} = \max(\text{wm},\ \min_{\text{active p}} \text{max_event}(p) - \text{ooo}) \cdot \text{fire} \iff \text{end} \le \text{wm} \cdot \text{drop} \iff \text{end} + \text{lateness} \le \text{wm}$$

SRC/NEXUS/STREAM.PY — FLINK SEMANTICS

```
    def _watermark(self, now: int) -> int:
        active = [self.s.part_max[p] for p, seen in self.s.part_seen.items() if now - seen
< IDLE]
        if active:
            wm = min(active) - self.ooo
        elif self.s.part_max:           # every partition idle: let processing time carry
the watermark (a heartbeat)
            wm = now - self.ooo - IDLE
        else:
            wm = NEG
        return max(self.s.wm, wm)

    def _emit(self, key, w: dict, now: int):
        value = {"n": w["n"], "sum": round(w["sum"], 4)}
        old = self.sink.get(key)
        self.sink[key] = {"value": value, "first_emit": old["first_emit"] if old else now,
"last_emit": now,
                          "emits": (old["emits"] if old else 0) + 1}
        self.emits.append((now, key[0] + self.size))

    def _advance(self, now: int):
        wm = self._watermark(now)
        if wm <= self.s.wm and self.s.wm != NEG:
            return
        self.s.wm = wm
        if not self.windowed:
            return
        done = wm - wm % self.size
        if not self.progress or done > self.progress[-1][1]:
```

THE RULES

- **Fire** when the watermark passes a window's end; **re-fire** on a late record within the hour; past it, the **side output**.
- **Dedupe** on event_id, kept 24 hours. **Dead letters** for what can't be read, with the reason.
- Offsets, windows, dedupe state and the ledger are **checkpointed together**; sinks are keyed, so a restore re-sends without changing anything.

IN THE RUN

- A TaskManager lost at 09:17: restored from the 09:10 checkpoint, **55** windows re-sent, sink **identical**.
- Ledger, page_views: read 22,813 = 22,509 counted + 112 duplicates + 192 side output.

Fast first. Right by morning.

The nightly batch recomputes every window from the whole topic. Every difference must be explained by the stream's side output.

TOPIC	WINDOWS	EQUAL	EXPLAINED	UNEXPLAINED	STREAM	BATCH	SIDE OUTPUT	SPARK AGREES
page_views	8,764	8,605	159	**0**	22,509	22,701	192	identical
partner_clicks	984	983	1	**0**	1,245	1,246	1	identical

The equation the run checks: batch events − stream events = side output, for every topic and every window. The PySpark batch (Spark 3.5.4) produces the same windows as the plain-Python one.

Volume hold: 2026-10-10 03:00 had 1,515 views where 29.0 is usual (z = 420.3). Held from Gold; three users over 120 views an hour removed; re-checked; released.

Write it once. Mask it everywhere.

contracts/sources.yml names the personal fields. The DDL, MERGE, masking and erasure for both warehouses are generated from it and committed; CI fails if they drift.

WAREHOUSE/SNOWFLAKE/REVIEWS_CURRENT.SQL — GENERATED

```
MERGE INTO NEXUS.GOLD.REVIEWS_CURRENT t
USING NEXUS.STAGE.REVIEWS_CURRENT s
  ON t.review_id = s.review_id
WHEN MATCHED AND s.erased AND s.version > t.version THEN DELETE
WHEN MATCHED AND NOT s.erased AND s.version > t.version THEN UPDATE SET location_id =
s.location_id, rating = s.rating, status = s.status, lang = s.lang, version = s.version,
_loaded_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED AND NOT s.erased THEN INSERT (review_id, location_id, rating, status,
lang, version, _loaded_at)
  VALUES (s.review_id, s.location_id, s.rating, s.status, s.lang, s.version,
CURRENT_TIMESTAMP());
```

SAME PRODUCT, TWO DIALECTS

- **Layout:** CLUSTER BY (TO_DATE(window_start), partner) · PARTITION BY DATE(window_start) CLUSTER BY partner
- **Personal data:** masking policy, SHA2 unless PII_READER · policy tag + data masking rule
- **Erasure list:** FLATTEN(PARSE_JSON(:review_ids)) · UNNEST(@review_ids)

CHECKED, NOT ONLY RENDERED

- Gold loaded with the same upsert semantics; **11/11** checks pass, including masking for every personal field and the erasure (5 rows removed).

AT LIFEPAL

- PII masking, role-based access and lineage in one governed layer; certified models behind an MCP server for agents.

partner_revenue_15m late for 79 minutes

Freshness is measured from the completeness the job publishes, so a quiet night is not an incident. One thing was all week.

UTC	WHO	WHAT
14:05	job	Record 3/171 can't be parsed. The job fails, restores from its checkpoint, reads the same record, and fails again.
14:08	would page	Restart count passes 3 in 10 minutes. Not alerted on yet: this is action 2.
14:35	SLA monitor	partner_revenue_15m is 50 minutes old, over its 45-minute SLA. Page sent.
14:39	on call	Acknowledged. Job RESTARTING; exception points to partition 3 offset 171.
15:20	on call	Mitigation: deploy with the dead-letter topic on. Restore from the last checkpoint; the bad record goes to the DLQ.
15:25	SLA monitor	Back within SLA. Every delayed window re-computed from the checkpoint.

29 min

to detect: freshness had to cross 45 minutes first

3 min

with a restart alert: 74 restarts said it early

Root cause: a record the job couldn't parse was treated as a reason to stop; a restart reads the same offset again. **Fix:** dead-letter by default in the shared template (done), page on restarts, alert on DLQ depth, a producer contract test.

Error budget for the week: 2.0 late samples allowed at 99.9%; the incident used 10. 11 windows late, none wrong; one record in the DLQ.

Python first. Java, checked.

The page_views job, in plain JDK 17 with no framework. CI compiles it, runs it on the same week, and fails if a single window differs from the Python job.

JAVA/SRC/NEXUS/WINDOWJOB.JAVA — JDK 17 · NO DEPENDENCIES

```
    void process(Rec r, long now) {
        count("read");
        Map<String, Object> v = Json.object(r.value());
        String id = (String) v.get("event_id");
        if (seen.containsKey(id)) {
            count("duplicates");
            return;
        }
        seen.put(id, now);
        long et = Instant.parse((String) v.get("event_time")).toEpochMilli();
        partMax.merge(r.partition(), et, Math::max);
        partSeen.put(r.partition(), now);
        long start = et - Math.floorMod(et, SIZE);
        Key key = new Key(start, (String) v.get("location_id"));
        if (start + SIZE + LATENESS <= wm) {
            count("late_dropped");
        } else {
            Window w = windows.computeIfAbsent(key, x -> new Window());
            w.n++;
            count("accepted");
            if (w.fired) {
                count("late_updates");
                emit(key, w);
            }
        }
        advance(now);
```

IN THE RUN

- **8,707** windows, **identical** to the Python job, in 584 ms.

STATED PLAINLY

- Kafka, Spark and Python are production experience. At Lifepal the streaming was Kafka + Debezium with Spark, not Flink; BigQuery is working level; Snowflake is from Deloitte.
- Java is the part I'd ramp up on. This is how I'd start: the same semantics, tested against a reference I trust.

How I'd start in Data Platform

Listen first, then own something small end to end, then make the template better for everyone.

DAYS 1–30

Learn the estate

- Read the SLAs and the last ten incidents
- Shadow on-call; map pipelines to owners
- Ship a small fix in Java and one in Python

DAYS 31–60

Own a pipeline

- Take one tier-1 pipeline end to end
- Freshness from completeness, not last write
- A runbook a new joiner can follow

DAYS 61–90

Improve the template

- Dead letters and restart alerts by default
- Contract → Snowflake and BigQuery SQL
- Stream/batch reconciliation as a standard check

QUESTIONS I'D BRING Which pipelines page most, and why? How are SLAs measured today? Where do Snowflake and BigQuery each fit? How much of streaming is Flink vs Spark?